

MediMind Project Technical Documentation (Start to Finish)

1. Executive Summary

MediMind is a Django-based hospital management platform built as a modular multi-app system. It supports patient, doctor, receptionist, lab-technician, and admin workflows across appointments, prescriptions, lab reports, payments, rooms, notifications, and AI-assisted decision support.

This document describes the system from architecture to modules, models, AI logic, APIs, data flow, security, and deployment status.

2. Technology Stack

Backend and Framework

- Python 3.x
- Django 6.0.3
- Django REST Framework
- django-cors-headers
- django-filter

Database and Storage

- PostgreSQL (configured as default runtime DB)
- Django ORM migrations
- Media storage for generated and uploaded files

AI/ML and Data

- scikit-learn
- pandas
- numpy
- pypdf

Document Generation

- reportlab (receipts, prescriptions, appointment docs, lab reports)

Communication and Integration

- Twilio SDK (WhatsApp notifications)
- SMTP email
- Khalti gateway keys/config path

Async and Performance Foundations

- Celery + Redis settings prepared

- WhiteNoise static serving
- GZip middleware enabled

3. Project Structure and App Responsibilities

- hms_project: settings, root URLs, WSGI/ASGI entry points.
- users: custom user model, profile management, auth, 2FA, dashboards, health records.
- clinical: doctor/specialization/schedule management.
- appointments: booking, waiting list, triage, report reader, no-show workflows.
- no_show_predictor: API-level no-show scoring logs.
- heart_risk: risk assessment workflow and ML model integration.
- drug_checker: interaction dataset + doctor API checker.
- prescriptions: prescription and refill lifecycle.
- lab: test templates, bookings, result management and report release.
- payments: payment lifecycle, receipt generation, refunds, insurance verification.
- rooms: admission and occupancy management.
- notifications: in-app and WhatsApp utility flows.
- reviews: patient review and ratings.
- audit: request/event trail support.

4. Authentication, Authorization, and Roles

User Model

Custom user entity uses email login and role-driven access:

- PATIENT
- DOCTOR
- RECEPTIONIST
- LAB_TECHNICIAN
- ADMIN

Security Controls

- Role checks in protected views.
- 2FA flow for sensitive roles.
- Login protection via failed attempt tracking.
- Rate-limit middleware for critical routes.
- Security headers middleware.
- Hardened cookie/session defaults.

5. Data Model Documentation

5.1 Users Domain

- User: identity, role, activation state.
- PatientProfile: demographic and contact profile.
- PatientHealthRecord: long-form clinical history notes.
- PatientVitalLog: periodic vitals history.
- LoginAttempt: brute-force lockout state.
- TwoFactorCode: OTP lifecycle and validation state.

5.2 Clinical Domain

- Specialization
- Doctor
- Shift
- DoctorSchedule
- DoctorLeave

5.3 Appointment Domain

- Appointment: date/time/status with patient-doctor mapping.
- WaitingList: overflow queue management.
- TriageAssessment: symptom-based priority and recommendations.
- MedicalReportAnalysis: uploaded report extraction + AI summary + risk flags.
- NoShowPredictor / NoShowPredictor_Summary: appointment no-show risk entities.
- NoShowPredictionLog (in no_show_predictor app): stored API prediction outcomes.

5.4 Prescription Domain

- Prescription
- PrescriptionItem
- PrescriptionRefill
- RefillReminder
- Pharmacy
- Additional drug entities in prescriptions/drug_models.py

5.5 Lab Domain

- TestTemplate
- TestField
- TestSchedule
- TestBooking
- TestResult
- TestResultItem

5.6 Payment Domain

- Payment
- Refund
- PaymentReminder
- PaymentReminderLog
- InsuranceVerification

5.7 Auxiliary Domain

- HeartRiskAssessment
- DrugInteraction
- InteractionCheckLog
- Notification
- Room, RoomBed, RoomAssignment, AdmissionRequest, RoomTransfer
- Review
- AuditLog

6. AI, ML, and Rule Engines

6.1 AI Triage (appointments/triage.py)

Input features:

- symptoms text
- duration_days
- pain_level
- emergency indicator booleans (fever, breathing issue, chest pain, bleeding, fainting)

Processing:

- Weighted score from symptom severity + keyword detection.
- Priority mapping to P1, P2, P3, P4.

Output:

- priority, priority_score, ai_summary, recommended_action

6.2 Medical Report Reader (appointments/report_reader.py)

Pipeline:

1. Read uploaded TXT/PDF.
2. Extract text (pypdf for PDF).
3. Detect report type (CBC, lipid, liver, renal, thyroid, diabetes, general).
4. Parse marker values using regex aliases.
5. Flag low/high/critical values using rule thresholds.

- 6. Produce risk level (LOW/MODERATE/HIGH/CRITICAL).
- 7. Generate 8+ point structured summary and recommendations.
- 8. Build personalized food/exercise/health guidance.

Safety and reliability:

- Encoding sanitization for legacy DB encodings.
- Fallback handling for unreadable files.
- Mandatory medical disclaimer in UI.

6.3 Heart Risk Assessment (heart_risk)

Inference strategy:

- ML-first prediction using persisted trained artifact.
- Fallback rule engine when model artifact unavailable.

Training source and model selection:

- Dataset fetched from OpenML (heart-disease v1).
- Candidate models: Logistic Regression, Random Forest, Gradient Boosting.
- Best model selected by ROC-AUC metric.
- Metadata persisted: metrics, model type, training timestamp, source info.

Output:

- risk_score (percent)
- risk_level (LOW/MEDIUM/HIGH)
- summary and recommendations
- factor narrative + metadata

6.4 No-Show Prediction

Implemented in two complementary layers:

- appointments/no_show_predictor.py model entities + heuristic prediction factors.
- no_show_predictor/utils.py API risk scoring for scheduling workflows.

Primary factors include:

- patient cancellation/missed history
- profile completeness
- doctor day load
- lead time to appointment
- weekend behavior signal

6.5 Drug Interaction Checker

Endpoint compares prescribed medicine combinations against stored interaction pairs.

Features:

- normalized name matching
- fallback partial-name match
- severity ranking
- interaction check logging

7. APIs and Endpoints

Representative JSON/API-style endpoints:

- GET appointments/api/slots/
- GET appointments/api/no-show-risk/
- GET heart-risk/api/search-patients/
- GET drug-checker/api/check-interactions/
- GET payments/insurance/
- POST payments/insurance/create/
- POST payments/insurance/<id>/verify/

Other module routes are template-rendered server-side views.

8. External Services and APIs

- Twilio WhatsApp API: reminder and alert messaging path.
- SMTP email: activation/reset communication.
- Khalti: payment key configuration for integration path.

LLM status clarification:

- No production external LLM API call is currently active in the checked implementation.
- Current AI features are implemented using deterministic rules and/or scikit-learn models.
- README roadmap may mention future LLM integration as planned work.

9. Document and Report Generation

Generated PDFs include:

- Appointment documents
- Prescriptions
- Lab reports
- Payment receipts

PDF characteristics:

- MediMind branding and legal footer metadata
- Patient and doctor context

- Hospital identity details and billing references

10. Data Flow (End-to-End)

Appointment to Payment

- 1. Patient books appointment.
- 2. Appointment status transitions through workflow.
- 3. Payment record auto-created on confirmation.
- 4. Reception/admin marks payment as paid.
- 5. Receipt PDF generated and downloadable.

Lab Workflow

- 1. Patient books lab test.
- 2. Technician fills and updates result fields.
- 3. Admin/authorized user verifies/release result.
- 4. Patient/doctor accesses released report and PDF.

AI Report Reader Workflow

- 1. User uploads lab/medical report file.
- 2. Text extraction and analysis run.
- 3. Risk/flags/summary saved in MedicalReportAnalysis.
- 4. Dashboard and detail pages show actionable interpretation.

11. Performance and SEO Enhancements

- Added gzip middleware for response compression.
- WhiteNoise static strategy for optimized static delivery.
- SEO metadata, robots.txt, sitemap.xml, and structured tags in base layout.
- Grouped navigation and dashboard visibility updates to improve discoverability.

12. Testing and Validation Approach

- Django system checks before and after major changes.
- Targeted app test execution for new features (especially report-reader and access control).
- Migration-backed schema updates with compatibility checks.

13. Configuration and Environment Variables

Important runtime variables include:

- SECRET_KEY
- EMAIL_HOST_USER / EMAIL_HOST_PASSWORD
- TWILIO_ACCOUNT_SID / TWILIO_AUTH_TOKEN / TWILIO_WHATSAPP_FROM

- KHALTI_PUBLIC_KEY / KHALTI_SECRET_KEY
- CELERY_BROKER_URL / CELERY_RESULT_BACKEND
- TWO_FACTOR and rate-limit related values

14. Current Delivery Status

Implemented and demonstrated:

- Core hospital workflows across appointments, prescriptions, lab, payments, rooms.
- Security middleware and role enforcement.
- AI-assisted modules (triage, report reader, heart risk, no-show, interaction checker).
- Professional document generation.

Remaining final-defense targets:

- Full production deployment packaging and cloud release steps.
- Real-time channel features and deeper analytics export polish.

15. Academic and Clinical Responsibility Notes

- AI outputs are for assistance and prioritization only.
- Final diagnosis/treatment decisions require licensed clinician review.
- The platform includes visible disclaimers in AI report-reader views.

16. Conclusion

MediMind is a strong modular healthcare platform with practical AI support, real workflow depth, and clear extensibility. The system has progressed from a standard HMS to a portfolio-grade architecture suitable for final defense completion with deployment and real-time enhancements.